FastAPI Reference

`Request` class

You can declare a parameter in a *path operation function* or dependency to be of type `Request` and then you can access the raw request object directly, without any validation, etc.

Read more about it in the FastAPI docs about using Request directly

You can import it directly from `fastapi`:

```
from fastapi import Request
```

> **Tip**
>
> When you want to define dependencies that should be compatible with both HTTP and WebSockets, you can define a parameter that takes an `HTTPConnection` instead of a `Request` or a `WebSocket`.

class fastapi.Request

```
Request(scope, receive=empty_receive, send=empty_send)
```

Bases: `HTTPConnection[StateT]`

Source code in `starlette/requests.py`

```
211 def __init__(self, scope: Scope, receive: Receive = empty_receive, send: Send = empty_send):
212     super().__init__(scope)
213     assert scope["type"] == "http"
214     self._receive = receive
215     self._send = send
216     self._stream_consumed = False
217     self._is_disconnected = False
218     self._form = None
```

attr scope `instance-attribute`

```
scope = scope
```

attr app `property`

```
app
```

attr url `property`

```
url
```

Ask AI

attr base_url `property`

```
base_url
```

attr headers `property`

```
headers
```

attr query_params `property`

```
query_params
```

attr path_params `property`

```
path_params
```

attr cookies `property`

```
cookies
```

attr client `property`

```
client
```

attr session `property`

```
session
```

attr auth `property`

```
auth
```

attr user `property`

```
user
```

attr state `property`

```
state
```

attr method `property`

```
method
```

attr receive `property`

```
receive
```

meth url_for

```
url_for(name, /, **path_params)
```

Source code in `starlette/requests.py`

```
def url_for(self, name: str, /, **path_params: Any) -> URL:
    url_path_provider: Router | Starlette | None = self.scope.get("router") or
self.scope.get("app")
    if url_path_provider is None:
        raise RuntimeError("The `url_for` method can only be used inside a Starlette application
or with a router.")
    url_path = url_path_provider.url_path_for(name, **path_params)
    return url_path.make_absolute_url(base_url=self.base_url)
```

meth stream `async`

```
stream()
```

Source code in `starlette/requests.py`

```
async def stream(self) -> AsyncGenerator[bytes, None]:
    if hasattr(self, "_body"):
        yield self._body
        yield b""
        return
    if self._stream_consumed:
        raise RuntimeError("Stream consumed")
    while not self._stream_consumed:
        message = await self._receive()
        if message["type"] == "http.request":
            body = message.get("body", b"")
            if not message.get("more_body", False):
                self._stream_consumed = True
            if body:
                yield body
        elif message["type"] == "http.disconnect":  # pragma: no branch
            self._is_disconnected = True
            raise ClientDisconnect()
    yield b""
```

meth body `async`

```
body()
```

Source code in `starlette/requests.py`

```
async def body(self) -> bytes:
    if not hasattr(self, "_body"):
        chunks: list[bytes] = []
        async for chunk in self.stream():
            chunks.append(chunk)
        self._body = b"".join(chunks)
    return self._body
```

meth json `async`

```
json()
```

Source code in `starlette/requests.py`

```
async def json(self) -> Any:
    if not hasattr(self, "_json"):  # pragma: no branch
        body = await self.body()
        self._json = json.loads(body)
    return self._json
```

meth form

```
form(
    *,
    max_files=1000,
    max_fields=1000,
    max_part_size=1024 * 1024
)
```

Source code in `starlette/requests.py`

```
def form(
    self,
    *,
    max_files: int | float = 1000,
    max_fields: int | float = 1000,
    max_part_size: int = 1024 * 1024,
) -> AwaitableOrContextManager[FormData]:
    return AwaitableOrContextManagerWrapper(
        self._get_form(max_files=max_files, max_fields=max_fields, max_part_size=max_part_size)
    )
```

meth close `async`

```
close()
```

Source code in `starlette/requests.py`

```
async def close(self) -> None:
    if self._form is not None:  # pragma: no branch
        await self._form.close()
```

meth is_disconnected `async`

```
is_disconnected()
```

Source code in `starlette/requests.py`

```
312 async def is_disconnected(self) -> bool:
313     if not self._is_disconnected:
314         message: Message = {}
315 
316         # If message isn't immediately available, move on
317         with anyio.CancelScope() as cs:
318             cs.cancel()
319             message = await self._receive()
320 
321         if message.get("type") == "http.disconnect":
322             self._is_disconnected = True
323 
324     return self._is_disconnected
```

meth send_push_promise `async`

```
send_push_promise(path)
```

Source code in `starlette/requests.py`

```
326 async def send_push_promise(self, path: str) -> None:
327     if "http.response.push" in self.scope.get("extensions", {}):
328         raw_headers: list[tuple[bytes, bytes]] = []
329         for name in SERVER_PUSH_HEADERS_TO_COPY:
330             for value in self.headers.getlist(name):
331                 raw_headers.append((name.encode("latin-1"), value.encode("latin-1")))
332         await self._send({"type": "http.response.push", "path": path, "headers": raw_headers})
```